ASSIGNMENT 3

Q1 - Dynamic Data Parser

JavaScript solution

```
// q1.js
"use strict";

const apiData = ["25", "true", "false", "NaN", " ", "100px", "3.14", null, undefined];
const validNumbers = [];
const invalidNumbers = [];

for (const value of apiData) {
  const numberValue = Number(value);
  const booleanValue = Boolean(value);
  const stringValue = String(value);

  // Empty/whitespace strings should not count as numeric data.
  const isBlank = typeof value === "string" && value.trim() === "";
  const isValidNumber = !isBlank && Number.isFinite(numberValue);

  console.log(`Original: ${stringValue} | Number: ${numberValue} | Boolean: ${booleanValue} | String: "${stringValue}"`);

  if (isValidNumber) {
    validNumbers.push(numberValue);
  } else {
    invalidNumbers.push(value);
    console.log(`  INVALID numeric value: ${stringValue}`);
  }
}

console.log("Valid numeric data:", validNumbers);
console.log("Invalid data:", invalidNumbers);
console.log(`Report: ${validNumbers.length} valid, ${invalidNumbers.length} invalid.`);
```

Explanation / Debug notes

Number() converts values to numbers; Number.isFinite() rejects NaN and infinities. A whitespace-only string is explicitly rejected because Number(' ') becomes 0. Boolean('false') is true because every non-empty string is truthy. null converts to 0 and undefined to NaN, so null is accepted as numeric by this code; change the validation rule if null/undefined must also be invalid.

Run the code in the browser console or Node.js. Save each solution in its own .js file as required.